

E-Commerce REST API with Role-Based Access (Django REST Framework)

Overview

This document defines the complete database schema and API structure for an **E-Commerce backend** built using **Django REST Framework (DRF)** with **role-based access control**. The project supports three user roles: **Super Admin**, **Admin**, and **User**.

The system includes features like product and category management, cart, orders, discounts, coupons, and product reviews.

User Roles

Role	Description	Permissions
Super Admin	Top-level administrator	Can manage Admins, Users, Categories, Products, Orders, and Discounts
Admin	Product and order manager	Can manage Categories, Products, Orders, and Discounts
User	Customer	Can browse products, manage cart, place orders, and post reviews

Database Schema

1. User (Django Default `auth_user`)

Field	Type	Description
id	PK	Unique user ID
username	String	Unique username
email	String	Unique email
password	String	Hashed password
first_name	String	First name
last_name	String	Last name
is_active	Boolean	Active flag
is_staff	Boolean	Used for admin access
date_joined	DateTime	Auto-created on registration

2. UserProfile

Field	Type	Description
id	PK	Profile ID
user	OneToOne → User	Linked user
phone_number	String	Optional phone
date_of_birth	Date	Optional
gender	String	Optional
profile_image	Image/URL	Optional
default_address	FK → Address (nullable)	Default address
role	Enum(SUPER_ADMIN, ADMIN, USER)	Defines user's access level

3. Address

Field	Type	Description
id	PK	Address ID
user	FK → User	Owner
name	String	Receiver name
street	String	Street details
city	String	City
state	String	State
country	String	Country
zipcode	String	Postal code
is_default	Boolean	Default address flag

4. Category

Field	Type	Description
id	PK	Category ID
name	String	Unique name
slug	Slug	URL-friendly
description	Text	Optional
parent	FK → self (nullable)	Hierarchical categories

5. Brand

Field	Type	Description
id	PK	Brand ID

Field	Type	Description
name	String	Brand name
description	Text	Optional
logo	Image/URL	Optional

6. Product

Field	Type	Description
id	PK	Product ID
name	String	Product name
slug	Slug	Unique
description	Text	Optional
category	FK → Category	Product category
brand	FK → Brand	Product brand
base_price	Decimal	Base price
stock	Integer	Stock count
created_by	FK → User	Admin/Super Admin who created the product
is_active	Boolean	Active or inactive flag

7. ProductImage

Field	Type	Description
id	PK	Image ID
product	FK → Product	Linked product
image_url	URL/Image	Product image

8. ProductVariant

Field	Type	Description
id	PK	Variant ID
product	FK → Product	Parent product
sku	String	Unique SKU code
name	String	Variant name (e.g., “Red - XL”)
price	Decimal	Variant price
stock	Integer	Variant stock

9. Discount

Field	Type	Description
id	PK	Discount ID
name	String	Discount name
discount_type	Enum(PERCENT, FIXED)	Discount type
value	Decimal	Discount amount/value
start_date	DateTime	Discount start date
end_date	DateTime	Discount end date
created_by	FK → User	Created by Admin/Super Admin
is_active	Boolean	Active flag

10. Coupon

Field	Type	Description
id	PK	Coupon ID
code	String	Unique coupon code
discount_type	Enum(PERCENT, FIXED)	Discount type
value	Decimal	Discount value
min_amount	Decimal	Minimum applicable order amount
expiry_date	DateTime	Coupon expiry date
usage_limit	Integer	Maximum usage count
created_by	FK → User	Created by Admin/Super Admin

11. Cart

Field	Type	Description
id	PK	Cart ID
user	FK → User	One cart per user
created_at	DateTime	Creation timestamp

12. CartItem

Field	Type	Description
id	PK	Cart item ID
cart	FK → Cart	Linked cart
product_variant	FK → ProductVariant	Variant added to cart
quantity	Integer	Quantity added
subtotal	Decimal	Calculated subtotal (quantity × price)

13. Order

Field	Type	Description
id	PK	Order ID
user	FK → User	Order owner
address	FK → Address	Delivery address
total_amount	Decimal	Total before discount
discount	Decimal	Discount applied
grand_total	Decimal	Final payable total
payment_status	Enum(PENDING, PAID, FAILED)	Payment status
order_status	Enum(PENDING, APPROVED, SHIPPED, DELIVERED, CANCELLED)	Order progress
created_at	DateTime	Order creation time
approved_by	FK → User (nullable)	Admin/Super Admin who approved the order

14. OrderItem

Field	Type	Description
id	PK	Order item ID
order	FK → Order	Linked order
product_variant	FK → ProductVariant	Product variant
quantity	Integer	Quantity ordered
price	Decimal	Snapshot of price during purchase

15. Review

Field	Type	Description
id	PK	Review ID
user	FK → User	Reviewer (role = USER only)
product	FK → Product	Reviewed product
rating	Integer (1–5)	Rating value
comment	Text	Optional comment
created_at	DateTime	Review creation time

16. Notification

Field	Type	Description
id	PK	Notification ID
user	FK → User	Recipient
title	String	Notification title
message	Text	Message content
is_read	Boolean	Read/unread flag
created_at	DateTime	Creation time

API Endpoints

1. Authentication and User Management

Endpoint	Method	Role	Description
/auth/register/	POST	Public	Register new user
/auth/login/	POST	Public	Obtain JWT token
/auth/logout/	POST	Authenticated	Logout user
/auth/profile/	GET/PUT	User/Admin	View or update profile
/auth/users/	GET	Super Admin	List all users
/auth/create-admin/	POST	Super Admin	Create new Admin

2. Category and Brand Management

Endpoint	Method	Role	Description
/categories/	GET	All	List categories
/categories/	POST	Admin/Super Admin	Create category
/categories/{id}/	PUT/DELETE	Admin/Super Admin	Update or delete category
/brands/	GET	All	List brands
/brands/	POST	Admin/Super Admin	Create brand

3. Product Management

Endpoint	Method	Role	Description
/products/	GET	All	List products with filters
/products/	POST	Admin/Super Admin	Create product
/products/{id}/	GET	All	View product details

Endpoint	Method	Role	Description
/products/{id}/	PUT/DELETE	Admin/Super Admin	Update or delete product
/products/{id}/variants/	POST	Admin/Super Admin	Add product variants
/products/{id}/images/	POST	Admin/Super Admin	Upload product images

4. Cart Management

Endpoint	Method	Role	Description
/cart/	GET	User	Retrieve cart items
/cart/add/	POST	User	Add item to cart
/cart/update/{item_id}/	PUT	User	Update quantity
/cart/remove/{item_id}/	DELETE	User	Remove item

5. Orders

Endpoint	Method	Role	Description
/orders/create/	POST	User	Create order from cart
/orders/	GET	User/Admin/Super Admin	List orders (Admin sees all)
/orders/{id}/	GET	User/Admin/Super Admin	Get order details
/orders/{id}/update-status/	PUT	Admin/Super Admin	Update order status
/orders/{id}/approve/	POST	Admin/Super Admin	Approve order

6. Discounts and Coupons

Endpoint	Method	Role	Description
/discounts/	POST	Admin/Super Admin	Create discount
/discounts/	GET	All	View active discounts
/coupons/	POST	Admin/Super Admin	Create coupon
/coupons/{code}/validate/	GET	User	Validate coupon code

7. Reviews

Endpoint	Method	Role	Description
/products/{id}/reviews/	GET	All	List product reviews
/products/{id}/reviews/	POST	User	Create review
/reviews/{id}/	DELETE	Admin/Super Admin	Delete review

8. Notifications

Endpoint	Method	Role	Description
/notifications/	GET	User/Admin	Fetch notifications
/notifications/{id}/mark-read/	POST	User/Admin	Mark as read
/notifications/send/	POST	Super Admin	Send notification manually

Technical Requirements

- Authentication**
Use JWT-based authentication (djangorestframework-simplejwt).
- Role-Based Access Control**
Implement custom DRF permission classes:
 - IsSuperAdmin
 - IsAdminOrSuperAdmin
 - IsUserOnly
- Custom UserProfile Model**
The role field is stored in UserProfile, not in the auth_user table.
- Nested Serializers**
Use nested serializers for Product → Variant → Images.
- Signals**
 - Reduce stock after order confirmation.
 - Send notifications on order updates.
- Filtering and Pagination**
Enable filtering and pagination for product listings.
- Logging Middleware**
Log API hits with user details and role information.

API Endpoints, Payloads & Responses

Authentication and User Management

1. Register User

Endpoint: POST /auth/register/
Access: Public

Payload

```
{
  "username": "ankush123",
  "email": "ankush@example.com",
  "password": "Pass@123",
  "first_name": "Ankush",
  "last_name": "Nag",
  "phone_number": "9876543210",
  "role": "USER"
}
```

Response

```
{
  "id": 1,
  "username": "ankush123",
  "email": "ankush@example.com",
  "role": "USER",
  "message": "User registered successfully."
}
```

2. Login

Endpoint: POST /auth/login/

Access: Public

Payload

```
{
  "username": "ankush123",
  "password": "Pass@123"
}
```

Response

```
{
  "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",
  "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",
  "user": {
    "id": 1,
    "username": "ankush123",
    "email": "ankush@example.com",
    "role": "USER"
  }
}
```

3. User Profile

Endpoint:

- GET /auth/profile/ → View profile
- PUT /auth/profile/ → Update profile

Access: Authenticated

PUT Payload

```
{
  "phone_number": "9998887776",
  "date_of_birth": "1998-07-15",
  "gender": "Male"
}
```

Response

```
{
  "user": {
    "id": 1,
    "username": "ankush123",
    "email": "ankush@example.com"
  },
  "profile": {
    "phone_number": "9998887776",
    "gender": "Male",
    "role": "USER"
  }
}
```

4. Create Admin (Super Admin only)

Endpoint: POST /auth/create-admin/

Access: Super Admin

Payload

```
{
  "username": "admin01",
  "email": "admin@example.com",
  "password": "Admin@123",
  "role": "ADMIN"
}
```

Response

```
{
  "id": 3,
  "username": "admin01",
  "email": "admin@example.com",
  "role": "ADMIN",
  "message": "Admin account created successfully."
}
```

Category and Brand Management

1. Create Category

Endpoint: POST /categories/

Access: Admin/Super Admin

Payload

```
{
  "name": "Electronics",
  "description": "All electronic items"
}
```

Response

```
{
  "id": 1,
  "name": "Electronics",
  "slug": "electronics",
  "description": "All electronic items"
}
```

2. List Categories

Endpoint: GET /categories/

Access: All

Response

```
[
  {
    "id": 1,
    "name": "Electronics",
    "slug": "electronics"
  },
  {
    "id": 2,
    "name": "Mobiles",
    "slug": "mobiles",
    "parent": 1
  }
]
```

3. Create Brand

Endpoint: POST /brands/

Access: Admin/Super Admin

Payload

```
{
  "name": "Samsung",
  "description": "Leading electronics brand"
}
```

Response

```
{
  "id": 1,
  "name": "Samsung",

```

```
    "description": "Leading electronics brand"
}
```

Product Management

1. Create Product

Endpoint: POST /products/

Access: Admin/Super Admin

Payload

```
{
  "name": "iPhone 15",
  "description": "Latest Apple phone",
  "category": 2,
  "brand": 1,
  "base_price": 99999,
  "stock": 20
}
```

Response

```
{
  "id": 1,
  "name": "iPhone 15",
  "slug": "iphone-15",
  "price": 99999,
  "stock": 20,
  "is_active": true
}
```

2. List Products

Endpoint: GET /products/

Access: All

Response

```
[
  {
    "id": 1,
    "name": "iPhone 15",
    "price": 99999,
    "category": "Mobiles",
    "brand": "Apple",
    "stock": 20
  }
]
```

3. Add Product Variant

Endpoint: POST /products/{id}/variants/

Access: Admin/Super Admin

Payload

```
{
  "sku": "IPH15-BLK-128",
  "name": "iPhone 15 Black 128GB",
  "price": 99999,
  "stock": 10
}
```

Response

```
{
  "id": 5,
  "sku": "IPH15-BLK-128",
  "name": "iPhone 15 Black 128GB",
  "price": 99999,
  "stock": 10
}
```

Cart

1. Add Item to Cart

Endpoint: POST /cart/add/

Access: User

Payload

```
{
  "product_variant": 5,
  "quantity": 2
}
```

Response

```
{
  "id": 3,
  "product_variant": {
    "id": 5,
    "name": "iPhone 15 Black 128GB",
    "price": 99999
  },
  "quantity": 2,
  "subtotal": 199998
}
```

2. View Cart

Endpoint: GET /cart/

Access: User

Response

```
{
  "user": "ankush123",
  "items": [
    {
      "product": "iPhone 15 Black 128GB",
      "quantity": 2,
      "subtotal": 199998
    }
  ],
  "total": 199998
}
```

Orders

1. Create Order

Endpoint: POST /orders/create/

Access: User

Payload

```
{
  "address": 1,
  "coupon_code": "WELCOME10"
}
```

Response

```
{
  "id": 10,
  "total_amount": 199998,
  "discount": 10000,
  "grand_total": 189998,
  "order_status": "PENDING",
  "payment_status": "PENDING"
}
```

2. Update Order Status

Endpoint: PUT /orders/{id}/update-status/

Access: Admin/Super Admin

Payload

```
{
```

```
  "order_status": "SHIPPED"
}
```

Response

```
{
  "id": 10,
  "order_status": "SHIPPED",
  "updated_by": "admin01"
}
```

Reviews

1. Add Review

Endpoint: POST /products/{id}/reviews/

Access: User

Payload

```
{
  "rating": 5,
  "comment": "Excellent phone!"
}
```

Response

```
{
  "id": 1,
  "user": "ankush123",
  "rating": 5,
  "comment": "Excellent phone!",
  "created_at": "2025-10-30T10:30:00Z"
}
```

Notifications

1. Send Notification

Endpoint: POST /notifications/send/

Access: Super Admin

Payload

```
{
  "user": 1,
  "title": "Order Approved",
  "message": "Your order #10 has been approved."
}
```

Response

```
{
  "id": 15,
  "title": "Order Approved",
  "message": "Your order #10 has been approved.",
  "is_read": false
}
```

Technical Implementation Notes

1. **Authentication:** JWT (djangorestframework-simplejwt)
2. **Permissions:** Custom DRF classes
 - o IsSuperAdmin
 - o IsAdminOrSuperAdmin
 - o IsUserOnly
3. **Pagination & Filtering:** DRF built-in support for product list